

ROTEIRO E PASSO A PASSO

Empacotamento de Código para Reuso – HR_Tech

Disciplina: Reuso de Software

Autor: Andryus Zolet

1. Objetivo do vídeo

Demonstrar, passo a passo, a aplicação de uma técnica de empacotamento de código focada em reuso no projeto HR_Tech. A demonstração deve evidenciar como uma parte do código pode ser organizada como um componente reutilizável, com dependências, metadados, versão, documentação e mecanismo de instalação/consumo.

O material da aula define empacotamento como o processo de transformar código-fonte e seus artefatos em um artefato distribuível, e destaca que o empacotamento não é apenas compactar arquivos, mas também formalizar contrato/API, metadados, versão, dependências e preparação para instalação e consumo.

2. Preparação antes da gravação

- Abrir o projeto HR_Tech no VS Code.
- Abrir um terminal na pasta raiz do projeto.
- Confirmar que o Composer está instalado.
- Deixar o GitHub do projeto disponível no navegador.
- Confirmar que o sistema e o banco de dados estão funcionando.
- Criar uma branch específica para a demonstração.

Comando sugerido para a branch:

```
git checkout -b compactacao-reuso
```

Depois, criar um ponto de restauração antes das alterações:

```
git add .  
git commit -m "Empacotamento Reuso"
```

3. Roteiro do vídeo

Cena 1 – Apresentação

O que mostrar: Mostrar o projeto HR_Tech no VS Code.

Fala sugerida: “Olá, neste vídeo iremos demonstrar a técnica de empacotamento de código focada em reuso aplicada ao projeto HR_Tech.”

Cena 2 – Explicar o que será reutilizado

O que mostrar: Mostrar os repositórios/CRUDs escolhidos para a demonstração, como EmployeeRepository, DepartmentRepository e BenefitRepository, conforme a estrutura real do projeto.

Fala sugerida: “Identificamos operações de acesso ao banco com comportamentos semelhantes. Vamos centralizar essa funcionalidade em um componente reutilizável.”

Cena 3 – Mostrar o problema

O que mostrar: Abrir dois ou mais arquivos que possuam lógica semelhante.

Fala sugerida: “A repetição dessa lógica aumenta a manutenção. Se uma regra mudar, vários arquivos poderiam precisar ser alterados.”

Cena 4 – Criar o componente reutilizável

O que mostrar: Criar BaseRepository.php e colocar nele as operações comuns que realmente existirem no projeto.

Mostrar a classe, a conexão PDO e métodos reutilizáveis como findAll e findById, adaptando os nomes das tabelas e campos ao HR_Tech.

Cena 5 – Fazer os componentes utilizarem a classe

O que mostrar: Alterar os repositórios concretos para reutilizarem BaseRepository.

Fala sugerida: “Agora os repositórios específicos reutilizam a implementação comum, evitando duplicação.”

Cena 6 – Criar/configurar o Composer

O que mostrar: Abrir ou criar composer.json.

Mostrar name, description, type, version, require e autoload. Explicar que esses metadados ajudam a transformar o componente em uma biblioteca reutilizável.

Cena 7 – Instalar e gerar autoload

O que mostrar: Executar composer install e composer dump-autoload.

Fala sugerida: “Agora vamos preparar o autoload para que as classes do componente sejam carregadas automaticamente.”

Cena 8 – Criar teste de consumo

O que mostrar: Criar um pequeno arquivo de teste que carregue vendor/autoload.php e utilize uma classe do componente.

Demonstrar a instanciação do repositório e uma operação real do banco.

Cena 9 – Executar o teste

O que mostrar: Executar php teste-reuso.php e mostrar o resultado.

Fala sugerida: “O componente está sendo consumido através do autoload e sua funcionalidade está funcionando.”

Cena 10 – Demonstrar o reuso

O que mostrar: Mostrar como o mesmo componente poderia ser consumido por outro projeto sem copiar manualmente suas classes.

Fala sugerida: “O objetivo do empacotamento é permitir que o componente seja instalado e utilizado por outros projetos.”

Cena 11 – Mostrar versionamento

O que mostrar: Mostrar a versão 1.0.0 no composer.json.

Fala sugerida: “A versão permite controlar alterações e compatibilidade entre diferentes versões do componente.”

Cena 12 – Mostrar documentação

O que mostrar: Abrir README.md e mostrar requisitos, instalação, utilização e versão.

Fala sugerida: “A documentação facilita a compreensão e adoção do componente por quem pretende reutilizá-lo.”

Cena 13 – Mostrar Git/GitHub

O que mostrar: Executar git status, git add ., commit e push. Depois abrir a branch no GitHub.

Mostrar que o trabalho foi versionado e está disponível no repositório.

Cena 14 – Conclusão

O que mostrar: Mostrar rapidamente a estrutura final.

Fala sugerida: “Transformamos uma parte do HR_Tech em um componente preparado para reuso, com código organizado, dependências, versão, documentação e mecanismo de consumo.”

4. Exemplo de composer.json

```
{
  "name": "c4pivara/hr-tech-reusable",
  "description": "Componentes reutilizáveis do projeto HR Tech",
  "type": "library",
  "version": "1.0.0",
  "require": {
    "php": ">=8.3"
  },
  "autoload": {
    "psr-4": {
      "HRTech\\": "src/"
    }
  }
}
```

5. Comandos utilizados no vídeo

```
git checkout -b compactacao-reuso
```

```
git status  
  
git add .  
  
git commit -m "Estado inicial antes do empacotamento"  
  
composer install  
  
composer dump-autoload  
  
php teste-reuso.php  
  
git add .  
  
git commit -m "Implementa empacotamento para reuso"  
  
git push origin compactacao-reuso
```

6. Estrutura visual para explicar no vídeo

Antes:

Código do HR_Tech → lógica repetida → manutenção em vários locais

Depois:

HR_Tech → componente reutilizável → Composer → versão/documentação/dependências → consumo por outros projetos

7. Perguntas que o professor pode fazer

O que vocês fizeram?

Transformamos uma parte do código do HR_Tech em um componente preparado para reuso, utilizando o Composer para organizar metadados, dependências e autoload.

Por que isso é reuso?

Porque a funcionalidade pode ser utilizada novamente sem copiar manualmente o código para cada projeto.

O que é empacotamento?

É transformar código e seus artefatos em um artefato distribuível, preparado para instalação e consumo.

Por que usar Composer?

Para gerenciar o pacote, dependências e autoload, facilitando a instalação e utilização do componente.

Qual a vantagem?

Redução de duplicação, manutenção centralizada e possibilidade de reutilização em diferentes projetos.

Como vocês controlam versões?

Por meio da versão definida para o componente, iniciando em 1.0.0.

8. Checklist antes de entregar o vídeo

- Projeto HR_Tech funcionando.
- Problema de duplicação identificado e explicado.
- Componente reutilizável criado.
- composer.json configurado.
- Dependências/requisitos definidos.
- Autoload funcionando.
- Versão definida.
- Teste de consumo executado.
- Reuso demonstrado.
- README/documentação apresentada.
- Commit realizado.
- Alterações enviadas ao GitHub.
- Todos os integrantes sabem explicar o que foi feito.